is_contiguous_layout

ISO/IEC JTC1 SC22 WG21 P0258R0 - 2016-02-12
Michael Spencer <bigcheesegs@gmail.com>
Audience: EWG, LEWG

Introduction

This paper proposes adding a new compiler-backed type trait to enable portably hashing an object as a range of bytes.

```
template <class T> struct is_contiguous_layout;
```

Usage

```
struct Trivial {
  char *Data;
  unsigned Len;
};

// Assert that Trivial can be hashed as a range of bytes if it is contiguous layout.
namespace std {
  template <>
  struct is_uniquely_represented<Trivial>
    : is_contiguous_layout<Trivial> {};
}
```

Motivation

P0029 proposes the user specialized trait `is_uniquely_represented` to assert that an object of a given type can be hashed by hashing its object representation. It states that `is_uniquely_represented` can only be true for types which have no padding bits, but intentionally avoids adding a type trait to check this to keep the proposal a "pure library" proposal.

It is not possible for users to portably assert that a type has the `is_uniquely_represented` trait without compiler assistance.

Trivial Example

```
struct Trivial {
  char *Data;
  unsigned Len;
};
```

The contiguous hashibility of this type depends on:

1. The `sizeof` `Data`, `Len` and `Trivial`. `sizeof(Data) + sizeof(Len)` must equal `sizeof(Trivial)`.
2. The implementation defined mapping between pointer values and their object representation.
3. The implementation defined mapping between unsigned values and their object representation.

1 can be checked via a static assert, however this is verbose.

2 and 3 can only be checked by reading the documentation for your implementation for every platform you intend to target, thus limiting portability.

Bitfields

```
struct Bitfields {
  unsigned char A : 4;
  unsigned char B : 4;
  unsigned short C : 8;
};
```

The contiguous hashibility of this type depends on how bitfields are implemented. In practice this is different between the Itanium ABI and Visual C++. This can also only be checked by taking a look at the docs.

Design Implications

The proposed wording for this trait is:

> A *contiguous-layout* type is a *standard-layout* type for which all bits in the object representation participate in the value representation (3.9).

This means that a standard-layout type is a contiguous-layout type unless there exist two object representation bit patterns of the type for which there exists no well defined program operating only on value representations to observe the difference.

Floats

Floating point values may compare equal even when they have different object representations. However this does not preclude them from having contiguous-layout types, as equality is not the only way to observe the difference between floating point values. The implementation may provide enough tools to observe the full state of the value.

It is implementation defined if floating point types are contiguous-layout.

Unions

Unions with members which are all the same size and are all contiguous-layout types are also obvious candidates for contiguous-layout types. However, unions with members of different sizes present a problem, as the existence of padding bits depends on the active member.

For union members which do not share a common initial sequence, implementations are allowed to omit copying the entire object representation of a union if it knows the active member. It is completely reasonable for an implementation to do this if a member was just assigned.

It is implementation defined if union types are contiguous-layout.

Proposed Wording

Proposed wording changes to the standard.

3.9 Types [basic.types]

Add a new paragraph.

> A *contiguous-layout* type is a *standard-layout* type for which all bits in the object representation participate in the value representation (3.9).

20.10.2 Header `<type_traits>` synopsis [meta.type.synop]

Add to synopsis.

```
template <class T> struct is_contiguous_layout;
```

20.10.4.3 Type properties [meta.unary.prop]

Add to table 49 - Type property predicates.

Template	Condition	Preconditions
`template <class T> struct is_contiguous_layout;`	T is a contiguous-layout type (3.9)	T shall be a complete type, (possibly cv-qualified) void, or an array of unknown bound.

References

- P0029 - A Unified Proposal for Composable Hashing, Geoff Romer, Chandler Carruth
- N3333 - Hashing User-Defined Types in C++1y, Jeffrey Yasskin, Chandler Carruth